Controlled Generative Development: A Deterministic Structural Core for AI-Native Software Design

Alexander Fedorenko
Independent Researcher
alex.grig.fed@gmail.com

March 2026

Abstract

This paper proposes Controlled Generative Development (CGD), a contract-first framework for AI-assisted development in which the specification precedes code, governs generation, and outlives code as the canonical artifact. The central object of CGD is the formal triple $T = \langle Y, G, \Sigma \rangle$, where Y is expressed through YASF, a minimal authoring layer in controlled natural language, G is a typed directed graph of functions and tables, and Σ is a set of table signatures. The paper formalizes the six-component function contract, subgraph contract computation, decomposition and aggregation rules, task-driven context assembly, and verification-driven dialogue. CGD is positioned as a contract-first approach between prompt-first, workflow-first, spec-first, and formal-first traditions. Using an end-to-end user registration case, we show that substantial structural defects can be detected and resolved before code generation, and that specification closure arises as a state of the entire multi-level loop rather than of any single layer. The paper is intentionally framed as conceptual and architectural: the formal core is defined here, while the reference implementation and empirical validation are placed on the immediate roadmap.

Keywords: controlled generative development; contract-first development; AI-native software design; typed dependency graphs; specification closure; verification-driven dialogue; task-driven context assembly

1 Introduction

Generative AI has sharply reduced the cost of writing code, but it has not solved the more fundamental problem: how to preserve the causal link between the author's original intent, the structure of the system, and the generated implementation. In most practical scenarios, an LLM receives either a free-form prompt or weakly structured context and then produces code that is verified only after the fact: by tests, manual review, or operation in production. This means that structural defects are detected too late—after they have already been embodied in code, dependencies, and data [4, 9].

In this paper we propose **Controlled Generative Development (CGD)**, an approach in which the specification precedes code, guides generation, and outlives code as the canonical artifact. The central object of CGD is the formal triple

$$T = \langle Y,\ G,\ \Sigma \rangle,$$

where Y is a YASF specification in controlled natural language, G is a typed directed graph of functions and tables, and Σ is a set of table signatures. The triple is treated not as documentation

but as a compilable and verifiable project invariant: changes in one component either cascade consistently into the others or are flagged by the verifier as a mismatch.

CGD does not claim the status of a main-track empirical paper with already demonstrated industrial effectiveness. This is a conceptual and architectural paper with a formal core, explicit limitations, and a roadmap toward a reference implementation. We deliberately do not claim strong operational metrics here. Instead, the paper answers an earlier and more foundational question: what should the minimal formal layer look like if AI-assisted development is to become structurally controllable before code is generated?

The main contributions of the paper are as follows.

1. We introduce CGD as a distinct **contract-first** framework situated between prompt-first, workflow-first, spec-first, and formal-first approaches.
2. We define YASF as a minimal authoring format for specification and, at the same time, as a potential intermediate representation for larger systems.
3. We specify the graph model $G = (V, E, \mathrm{src}, \mathrm{dst}, \tau_V, \tau_E)$ with two node types, five edge types, and the six-component function contract $C(f) = \langle C_{in}, C_{out}, R, W, T_{in}, T_{out} \rangle$.
4. We formalize subgraph decomposition and aggregation with a computable correctness condition via ComputeSubgraphContract and `refinement_map`.
5. We define **task-driven context assembly** as a deterministic function of the contractual neighborhood of the target node.
6. We close the cycle through **contract-driven generation** and **reverse verification**, while explicitly distinguishing what is already formalized from what remains on the roadmap.
7. We demonstrate the pre-generation part of the cycle on a single end-to-end user registration case in five examples.
8. We introduce **verification-driven dialogue** as an infrastructural loop in which the verifier not only finds defects but also generates targeted questions for the user until specification closure is reached.

2 Existing Approaches to AI-Assisted Development

2.1 Classification of approaches

Existing approaches to AI-assisted development can be usefully classified along two axes: the degree of specification formalization and the point at which verification takes place. For the purposes of this paper we distinguish **five** categories.

Prompt-first. The developer formulates the task in free form and the LLM generates code. There is no independent specification artifact; the prompt is ephemeral and poorly reproducible. Verification is entirely post-generative [3, 4].

Workflow-first. Generation is organized as a sequence of roles, agents, or stages. The process becomes more structured, but the data, dependencies, and contracts are still passed as text between steps. Verification is still aimed at the result rather than at a formal model of the system [7, 15, 16].

Spec-first. The specification becomes the primary artifact: the developer maintains a structured text from which code and tests are generated. This is a major step forward because intent is separated from implementation. However, the architecture usually remains single-level: "specification → generation → tests → revise the specification." The closest industrial example to our work is **CodeSpeak**: textual `.cs.md` specs, code generation, mixed mode, and takeover for legacy code. This class of tools confirms that there is already industrial demand for specification-first practice [1, 6, 13, 14].

Formal-first. The system is described in a formal language with mathematical semantics—TLA+, Alloy, Lean, Z, VDM, and similar systems. The guarantees are maximal, but the cost of entry is high, and direct integration with LLM-based code generation is usually not the primary goal [8, 10].

Contract-first (CGD). The developer writes in the minimal YASF format, which is deterministically mapped into the triple $T = \langle Y, G, \Sigma \rangle$. Structural and contractual verification is performed **before** code generation. The LLM context is assembled from the formal neighborhood of a node rather than from a global bag of text. The architecture is multi-level: different classes of defects are filtered at different infrastructure levels.

2.2 Evolutionary axis

These categories form a working evolutionary axis in terms of increasing formalization and depth of control:

Prompt-first → Workflow-first → Spec-first → Contract-first (CGD).

At each step a new stable layer appears. Workflow-first adds process structure. Spec-first introduces the specification as an explicit artifact. Contract-first adds a formal system model and pre-generative verification. Formal-first sits at the same end of the spectrum in terms of rigor, but forms a parallel branch: it is not the next evolutionary step of LLM-native development, but a separate tradition with a different balance between guarantees and entry cost [5, 7, 8, 10, 12, 15, 16].

2.3 CodeSpeak as the closest industrial analogue

CodeSpeak matters for this paper not as a competitor, but as evidence of engineering demand. It shows that a spec-first cycle based on `.cs.md` specifications, generation, mixed mode, and takeover is already perceived as a practical alternative to direct code editing. CGD, however, operates one level deeper: it introduces a formal multi-level scaffold in which the specification is not only an input to the generator, but also an object of structural verification before code exists [1, 6, 14].

In CGD terms, CodeSpeak primarily covers the authoring-specification level and part of the generation level, but it does not explicitly introduce a separate graph layer, a separate contract-and-signature layer, or verification-driven dialogue as an infrastructural loop. CGD is aimed precisely at this vacant niche [1, 6, 14].

3 Controlled Generative Development

3.1 The triple as an invariant

The central artifact of CGD is the triple $T = \langle Y, G, \Sigma \rangle$, where Y captures the semantic dimension of the specification, G its structural dimension, and Σ its typological dimension. These three dimensions are not strictly orthogonal: Y compiles into G, the graph refers to Σ, and Σ constrains

the admissible links in G. Nevertheless, they form three distinguishable coordinate axes of description, sufficiently independent to serve as a working basis for design.

Design Thesis 1 (Sufficiency of the triple)**.** *For the class of information systems with discrete data flows, the triple $\langle Y, G, \Sigma \rangle$ is sufficient to express and verify those aspects of a system that concern the structural organization of functions, data flows, and contractual compatibility. Full formal and empirical validation of this hypothesis remains future work.*

The determinism of compilation from YASF into graph and signatures is likewise stated as an **implementation invariant**, to be checked by conformance tests, rather than as an already established empirical fact.

3.2 Task-driven context assembly

The traditional way of working with LLMs looks like "collect as much context as possible, compress it, and hope the model uses exactly what matters." This strategy systematically creates two extremes: *context pollution* and *context loss* [11].

CGD replaces this pattern with **task-driven context assembly**. For a target function f, the context is defined not by dialogue history and not by an arbitrary RAG set, but by the formal neighborhood of its contract. In the basic case, the context includes:

- the full contract $C(f)$;
- the signatures of all tables in $C_{in}(f)$, $C_{out}(f)$, $R(f)$, $W(f)$;
- the contracts of first-order functions connected to f through $T_{in}(f)$ and $T_{out}(f)$;
- when necessary, the signatures of tables jointly used with first-order neighbors.

The context therefore becomes a deterministic function of the node and its contractual neighborhood. It is local, reproducible, and independent of how exactly the developer or agent happened to describe the project in earlier interactions.

3.3 Contract-driven generation, reverse verification, and the sterile zone

Once specification closure has been reached, the code generator receives not a free textual description but a strict technical assignment derived from the contract and signatures. The contract fixes inputs, outputs, reads, writes, and triggers. The table signatures fix fields, keys, kind, and constraints. This constrains the space of admissible implementations before the first token of code is generated.

Generation itself remains non-deterministic: the model may choose different algorithms, styles, and code structures. But any implementation is then compared not with the original prompt, but with the canonical triple.

This is also where the principle of the **sterile zone** is introduced. In CGD a project is considered structurally sterile as long as unambiguous traceability and reverse verifiability are preserved between Y, G, Σ, and code. Manual edits to generated code are treated as violations of sterility. Classical round-trip engineering is deliberately rejected as the default process model. If legacy or handwritten code must be included in the system, it goes through an **assimilation** procedure: decomposition, extraction of the actual triple T', reconstruction of YASF, and renewed achievement of closure [1].

3.4 Verification-driven dialogue

Verification-driven dialogue turns specification building from a monologue by the author into a controlled cycle:

build the triple → verify → classify defects → ask the user → refine the triple → repeat.

The verifier splits defects into three classes.

- **Blocking** — defects that cannot be resolved correctly without a user decision.
- **Auto-resolvable** — only safe normalizations that do not alter domain semantics: canonicalization of names, aliases, enum/PK/FK, kind, sections, and duplicate edges.
- **Advisory** — warnings that may require a human decision but do not block progress of the cycle itself.

Specification closure is therefore not a property of a single level and not a synonym for "the graph passed L2." It is a state of the whole loop, reached after the blocking queue is empty and all user decisions have cascaded downward through the specification.

3.5 Multi-level CGD infrastructure

3.5.1 Strategic principle

The strategic principle of CGD is simple: **make the infrastructure more complex, not the author**. The author should work with a minimal and naturally readable representation. Everything related to formal rigor, contractual algebra, graph checking, and context assembly is shifted into the infrastructural layer.

3.5.2 Five processing levels

CGD uses five levels.

L0 — user / interface. At this level there is only intent, expressed in free natural language.

L1 — authoring / YASF. At this level a partial triple $T_1 = \langle Y, -, - \rangle$ appears: a structured YASF block with a canonical subset of sections.

L2 — structural / graph. Here $T_2 = \langle Y, G, - \rangle$ appears: YASF compiles into a typed graph. Structural defects are filtered out—illegal edges, isolated nodes, duplicates, broken flows, and missing mandatory paths.

L3 — contractual / signatures. At this level the full triple $T_3 = \langle Y, G, \Sigma \rangle$ is formed. Signature compatibility, PK/FK integrity, kind, enum compatibility, and contract completeness are checked.

L4 — generation + reverse verification. The canonical triple is used for generation and then for reverse checking of the actual triple T' extracted from code.

3.5.3 Layered defect filtering and specification closure

Each level filters its own class of defects and thereby reduces the burden on the next one. A problem caught at L2 should not survive until L4. This is why closure is formulated as a state of the whole infrastructural loop.

In this paper **specification closure** is considered achieved when the following five conditions hold simultaneously:

1. all structural checks are green;
2. there are no unresolved references, signatures, or aliases;
3. all contracts are closed;
4. all mandatory paths terminate in an external effect;
5. the blocking-defect queue is empty.

3.5.4 Event routing: downward cascade and upward escalation

CGD uses two complementary trajectories.

Downward cascade: L0 $\rightarrow$ L1 $\rightarrow$ L2 $\rightarrow$ L3 $\rightarrow$ L4. Intent is progressively formalized.

Upward escalation: if L2 or L3 detects a blocking defect, it is raised to L0 as a targeted question to the user; the answer then cascades back down through a YASF update, graph recompilation, and contract recomputation.

3.5.5 Comparison with single-level approaches

Single-level spec-first tools mix semantic incompleteness, structural defects, contractual mismatches, and generator errors within one loop [1]. CGD separates these causes across responsibility levels. As a result, every defect receives an address: where it arose, where it must be resolved, and how the consequences of the resolution cascade.

3.5.6 The triple as a working basis of the specification space

The triple $\langle Y, G, \Sigma \rangle$ defines three distinguishable dimensions of the specification space:

- Y covers intent, processing rules, and branching conditions;
- G covers the topology of functions and data flows;
- Σ covers table typing and integrity constraints.

We treat this triple as a **working basis** and a constructive design thesis [2, 12]. This makes it possible to interpret the CGD levels as different projections of the same formal space and makes the architecture extensible: a new level can be added without revising the basis, provided that a mapping from the space T is defined for it [8, 10].

The extensibility principle has a practical integration consequence. Any external tool—a code generator, linter, CI/CD pipeline, documentation generator, or cross-project analyzer—can be integrated into CGD as an additional level, provided that it defines its own mapping from the space T. The triple $T = \langle Y, G, \Sigma \rangle$ serves as a common formal integration interface: tools connect through a formal projection contract rather than through an ad hoc API. In this sense, CGD is an integration platform, not an alternative to existing development tools.

3.5.7 Representation of control structures

Control flow is covered by the triple *representationally*, not in full operational semantics.

- In Y, branching, loops, and parallelism are described in the Processing section in controlled natural language.
- In G, these structures become topology: success-path and error-path, back-edges for loops, and independent branches for parallelism.
- In Σ, the data traveling along these paths are typed.

What remains outside the current formalization are the exact branch-selection rules, execution policies for loops, priorities of parallel branches, timing, and transactionality. This is deliberately deferred to the limitations section.

4 Formal Core

4.1 YASF

YASF (Yet Another Specification Format) is the minimal authoring layer of specification in controlled natural language. For the purposes of this paper, we use a **canonical subset of sections** sufficient to derive the graph and contracts: purpose, input data, reference data, processing, output data, errors, side effects, triggers, and contract.

It is important not to overstate the status of the format. YASF is not presented here as a forever fixed set of sections. For larger systems it may serve not only as something written by a human, but also as an internal **IR** generated by tooling. What remains invariant is not the origin of YASF, but its role as the semantic input to the triple.

Compilation YASF $\to (G, \Sigma)$ is treated as a deterministic implementation goal and should be checked by conformance tests.

4.2 Graph model and contracts

The graph is defined as

$$G = (V,\ E,\ \mathrm{src},\ \mathrm{dst},\ \tau_V,\ \tau_E),$$

where $\tau_V(v) \in \{\texttt{function},\ \texttt{table}\}$, and $\tau_E(e) \in \{\texttt{consumes},\ \texttt{produces},\ \texttt{reads},\ \texttt{writes},\ \texttt{triggers}\}$.

In the current synchronized version of the paper, edge types are defined as follows:

- `consumes`: table $\to$ function
- `produces`: function $\to$ table
- `reads`: table $\to$ function
- `writes`: function $\to$ table
- `triggers`: function $\to$ function

Table $\to$ table links are forbidden. Function $\to$ function links are permitted only for `triggers`. Triggers to tables are forbidden.

For each function f, the contract is defined as

$$C(f) = \langle C_{in},\ C_{out},\ R,\ W,\ T_{in},\ T_{out}\rangle,$$

where all components are derived directly from the graph [12].

4.2.1 Subgraph and boundary

Let $V_{\text{func}}(S) \subseteq V$ be a chosen set of functions. Then define the set of subgraph tables as

$$V_{\text{tables}}(S) = \{t \in V \mid \tau_V(t) = \texttt{table} \text{ and } t \text{ is incident to at least one edge connected to a function from } V_{\text{func}}(S$$

Then:

- $V(S) = V_{\text{func}}(S) \cup V_{\text{tables}}(S)$;
- $E(S) = \{e \in E \mid \text{src}(e) \in V(S) \text{ and } \text{dst}(e) \in V(S)\}$.

A table is considered internal if all its incident edges connect it only to functions from $V_{\text{func}}(S)$. The remaining tables form the boundary ∂S.

4.2.2 ComputeSubgraphContract

```
function ComputeSubgraphContract(S):
    Vfunc = V_func(S)
    Vtabs = V_tables(S)

    Internal = { t in Vtabs |
                 every edge incident to t
                 is incident only to functions in Vfunc }
    Boundary = Vtabs - Internal

    Cin(S)  = { t in Boundary |
                exists consumes edge  t -> f, f in Vfunc }
    Cout(S) = { t in Boundary |
                exists produces edge f -> t, f in Vfunc }
    R(S)    = { t in Boundary |
                exists reads edge    t -> f, f in Vfunc }
    W(S)    = { t in Boundary |
                exists writes edge   f -> t, f in Vfunc }
    Tin(S)  = { g not in Vfunc |
                exists trigger g -> f, f in Vfunc }
    Tout(S) = { g not in Vfunc |
                exists trigger f -> g, f in Vfunc }

    return <Cin(S), Cout(S), R(S), W(S), Tin(S), Tout(S)>
```

This definition is consistent with the examples: external tables are present in $V(S)$ as boundary tables, and the subgraph contract is computed precisely from them rather than from abstract "interface tables" outside the induced subgraph.

4.2.3 Decomposition and aggregation

A function f may be replaced with a subgraph S if three conditions hold:

1. S is induced by its $V(S)$;
2. all external links pass only through ∂S and the trigger boundary;
3. the external contract, after applying the internalization/externalization rule, is preserved: $C(S) = C(f)$.

One clarification is needed here. If a function $g \in T_{out}(f)$ is included inside the subgraph, **trigger internalization** occurs: g is removed from $T_{out}(S)$, and its boundary outputs become part of $C_{out}(S)$. Aggregation performs the inverse operation—**externalization**. Thus the reversibility of decompose/aggregate becomes parameterized by the subgraph boundary, but is not broken.

Aggregation is admissible if the subgraph is collapsed into a node f_S with the same external contract and is accompanied by a `refinement_map` sufficient to recover the internal structure exactly.

4.3 Table signatures and compatibility

For each table t, the signature is defined as

$$\Sigma(t) = \langle \text{Fields, Keys, Kind, Constraints} \rangle.$$

In the current JSON profile, fields are typed by base types such as `string`, `integer`, `boolean`, `float`, `timestamp`, `date`, `uuid`, `text`, and `json`; enum and regex constraints are expressed via `Constraints`. The attribute `role` may be present as an auxiliary semantic hint, but in the current version it **does not participate in formal compatibility**.

The canonical kinds in the synchronized version are **entity, event, reference, log, projection, error**. For the purposes of the current version, every table, including `projection`, is required to have a canonical PK or an explicit surrogate key; this removes the contradiction between representation semantics and serialization rules.

Signature compatibility is checked after canonicalization. It is convenient to distinguish **strict equality** $\Sigma(A) = \Sigma(B)$ from **inclusion compatibility** $\Sigma(A) \supseteq \Sigma(B)$. It is important to state the direction of the enum rule explicitly. If a producer function emits a table that is then interpreted by a consumer function, then for every enum field the following must hold:

$$\text{Allowed(producer)} \subseteq \text{Allowed(consumer)}.$$

Otherwise the producer may generate a value that the consumer cannot interpret, which is an L3 defect.

In the current version of the paper, per-consumer field contracts are **not formalized**: a function operates on a table as a whole object, rather than on a formally specified projection of its fields.

5 End-to-End Case: User Registration

5.1 Example 1 — Birth of the triple

From the request "register a user by email, persist the record, send a confirmation, and log the event," the following base tables are extracted: **RegistrationRequest**, **User**, **RegistrationEvent**, **AuditLog**, **NotificationEvent**; and two functions: **RegisterUser** and **SendConfirmation**.

Top-level contract:

$$C(\text{RegisterUser}) = \langle \{\text{RegistrationRequest}\},\ \{\text{User, RegistrationEvent}\},\ \{\text{User}\},\ \{\text{AuditLog}\},\ \emptyset,\ \{\text{SendConfirm}$$

Trigger-function contract:

$$C(\text{SendConfirmation}) = \langle \{\text{User}\},\ \{\text{NotificationEvent}\},\ \emptyset,\ \emptyset,\ \{\text{RegisterUser}\},\ \emptyset \rangle.$$

The graph contains **2 functions, 5 tables, and 8 edges**.

5.2 Example 2 — Decomposition

The function **RegisterUser** is decomposed into **ValidateInput**, **CheckUniqueness**, **CreateUser**, and **SendConfirmation**. Two intermediate tables appear inside the subgraph: **ValidatedRequest** and **UniqueRequest**.

Applying ComputeSubgraphContract yields:

$$C(S) = \langle \{\text{RegistrationRequest}\},\ \{\text{User, RegistrationEvent, NotificationEvent}\},\ \{\text{User}\},\ \{\text{AuditLog}\},\ \emptyset,\ \emptyset \rangle.$$

This is consistent with the internalization rule: **SendConfirmation** has moved inside the subgraph, so it disappears from $T_{out}(S)$, while its boundary output **NotificationEvent** appears in $C_{out}(S)$.

At this point closure is still not reached: the success path is formalized, but the error path is missing. The subgraph contains **4 functions, 7 tables, and 12 edges**.

5.3 Example 3 — Defect detection and repair

At this step the verifier finds **2 blocking and 2 advisory** defects.

Blocking 1. ValidateInput has no error path.

Blocking 2. CheckUniqueness has no error path.

After two targeted questions, the user chooses a unified behavior: in both cases the function produces **RegistrationError**.

Advisory 1. There is no write into **AuditLog** on the error path. This is not an auto-resolvable case: the decision depends on the business audit policy. In the running case, the user decides **to add** AuditLog writes for the error path.

Advisory 2. RegistrationEvent has no consumer inside the current graph. The user explicitly states that the event is consumed by external analytics.

After the update, the subfunction contracts become closed, and the top-level contract is refined to version **v2**:

$$C(\text{RegisterUser}) = \langle \{\text{RegistrationRequest}\},\ \{\text{User, RegistrationEvent, RegistrationError}\},\ \{\text{User}\},\ \{\text{AuditLog}$$

Specification closure is reached here, not at the decomposition step. The subgraph now contains **4 functions, 8 tables, and 16 edges**.

5.4 Example 4 — Aggregation

The repaired subgraph is collapsed back into the single node **RegisterUser**. During aggregation, the inverse trigger externalization occurs: **SendConfirmation** once again becomes an external function in $T_{out}(\text{RegisterUser})$, and **NotificationEvent** moves back into **SendConfirmation**'s contract.

The external contract of the aggregated node matches **v2**. The `refinement_map` preserves the internal functions, intermediate tables, and internal edges.

It is important here to distinguish two levels of reversibility. At the topology level, the transformation decompose/aggregate is reversible. At the level of contract component distribution, reversibility is **parameterized by the subgraph boundary**: an internalized trigger moves its outputs into $C_{out}(S)$, and an externalized trigger returns them to the child function. This is not a side effect of an informal heuristic, but part of the transformation rule itself.

After aggregation, the graph contains **2 functions, 6 tables, and 9 edges**, while closure is preserved.

5.5 Example 5 — Model comparison, normalization, and convergence to the project canon

The same request is sent to two models.

- **Model A** returns a simplified graph with names such as `InputForm`, `UserRecord`, and `EmailLog`, without PK/kind and without an error path.
- **Model B** returns a more detailed structure with `RegistrationRequest`, `Users`, `Notification`, and `AuditLog`, but still without `RegistrationError` and `RegistrationEvent`.

Comparison is performed in three layers.

1. **Overall topology.** Both models reproduce the success path "input → processing → user creation → notification."
2. **Structural differences.** Names, signature completeness, and the presence of reads/writes differ.
3. **Deviations from the project canon.** Both models lack the error path and `RegistrationEvent`; Model A is additionally structurally incomplete.

Normalization does not mean that the models "converge by themselves." On the contrary, triples are extracted from both outputs, discrepancies are classified, and then both are normalized against the **project reference model** from Examples 1–4. The canonical result matches the **v2** contract.

After normalization, the graph has **2 functions, 6 tables, and 9 edges**.

5.6 Quantitative summary

All numbers in this section are **illustrative**, not benchmark data.

Example	Functions	Tables	Edges
1 — Birth	2	5	8
2 — Decomposition	4	7	12
3 — Defect repair	4	8	16
4 — Aggregation	2	6	9
5 — Normalization	2	6	9

Table 1: Graph metrics across the five running examples.

In terms of defects, the case yields **2 blocking + 2 advisory** defects and one iteration of verification-driven dialogue before closure is reached. For the function **CheckUniqueness**, task-driven context assembly reduces local context by approximately **65%** relative to the full graph; this number is illustrative and is not presented as a general empirical metric.

6 Verification Without Code

6.1 Pre-generative verification

Before code generation, the following are checked:

- type correctness of edges;
- uniqueness of identifiers and absence of duplicates;

- absence of isolated nodes;
- presence of PK and kind for every table;
- consistency of contracts with the graph;
- completeness of mandatory paths;
- canonicality of signatures and contractual compatibility.

The key effect of CGD is that a significant part of architectural defects becomes observable **before** code exists.

6.2 Post-generative verification

The post-generative part of the cycle is specified in this paper as a formal roadmap rather than as a completed reference implementation. The actual triple T' is to be extracted from code, after which T' is compared with the canonical T.

Some such discrepancies can be extracted strictly: extra dependencies, missing outputs, extra writes, altered signatures. Some remain analytical: hidden logic, non-formalized path assumptions, or unused fields in the absence of per-consumer field contracts.

7 Limitations and Roadmap

7.1 Current limitations

The current version does not support [2, 5, 10]:

- per-consumer field contracts;
- path assumptions as a formal contract component;
- contract subtyping;
- developed operational semantics for loops and parallelism;
- concurrent access and temporal annotations;
- schema migrations and a full-fledged runtime-monitoring layer.

7.2 Implementation status

At the time of this paper, YASF, graph v0.3, tables v0.2, JSON representations, and the running case are formally defined. A reference implementation of the validator, compiler, and reverse verification is not yet claimed as a production-ready system. This boundary is drawn consciously and explicitly.

7.3 Near-term steps

The roadmap includes:

- a reference implementation of the YASF $\rightarrow$ JSON-artifact compiler;
- a structural + contract consistency validator;
- a task-driven context generator;
- a reverse verification prototype;
- case expansion beyond the user-registration scenario;
- controlled evaluation on several real projects.

7.4 Threats to validity

Construct validity. The notions of structural defect, closure, and contractual compatibility are introduced by the authors and may fail to cover all practical error classes.

Internal validity. The running case is synthetic and small. It tests internal consistency of the model, not industrial performance.

External validity. Transferability to domains with complex concurrency, temporal logic, and rich state semantics has not yet been demonstrated.

Reliability. The quality of verification-driven dialogue depends not only on the formal model, but also on how well the infrastructure turns a defect into a clear and targeted question for the user [5, 9].

8 Conclusion

CGD offers not just another prompt template and not just another workflow around an LLM, but an earlier control layer—a layer at which the specification exists as a formal and verifiable invariant even before code is written [8].

The central result of the paper is the construction $T = \langle Y, G, \Sigma \rangle$ together with the associated multi-level infrastructure L0–L4. This infrastructure makes it possible to filter defects layer by layer, assemble task-oriented context from the contractual neighborhood, keep the project inside the sterile zone, and avoid delegating to the code generator those architectural decisions that should have been made earlier. The architecture is open to extension: new levels and external tools can be connected through a formal projection contract, turning CGD into an integration platform.

We deliberately state the completeness of the triple and the sufficiency of its basis as a **design thesis**, not as an already proven mathematical fact. The immediate next step is a reference implementation and controlled empirical validation. Even in its current form, however, CGD defines a clean architectural framework: the specification should precede code, govern generation, and outlive code as the canonical artifact.

References

[1] Andrey Breslav. CodeSpeak: Software engineering with AI. https://codespeak.dev/, 2026. Alpha version, February 2026.

[2] Chunhui Chen, Jichen Liu, Baojiang Wei, Cuiyun Zhou, and Lei Zhao. Application of knowledge graph in software engineering field. *Information and Software Technology*, 164: 107316, 2023.

[3] Cursor. Cursor: The AI code editor. https://cursor.com/, 2024.

[4] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Zhen Ming Jiang. GitHub Copilot AI pair programmer: Asset or liability? *Journal of Systems and Software*, 203:111734, 2023. arXiv:2206.15331.

[5] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.

[6] Habr. Architecture instead of syntax: CodeSpeak as a language of development. https://habr.com/ru/articles/1010082/, 2026. In Russian. 14 March 2026.

[7] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *Proceedings of ICLR 2024*, 2024. arXiv:2308.00352.

[8] Daniel Jackson. *Software Abstractions: Logic, Language, and Analysis*. MIT Press, revised edition, 2012.

[9] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. A survey on large language models for code generation. *ACM Transactions on Software Engineering and Methodology*, 2024. arXiv:2406.00515.

[10] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[11] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020. arXiv:2005.11401.

[12] Bertrand Meyer. *Object-Oriented Software Construction*. Prentice Hall, 2nd edition, 1997.

[13] Microsoft. TypeChat: Building natural language interfaces using types. https://github.com/microsoft/TypeChat, 2023.

[14] Gergely Orosz. The programming language after Kotlin – with the creator of Kotlin. The Pragmatic Engineer Newsletter, 12 February 2026. https://newsletter.pragmaticengineer.com/p/the-programming-language-after-kotlin, 2026.

[15] Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the ACL*, pages 15174–15186, 2024. arXiv:2307.07924.

[16] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation framework. *COLM 2024*, 2023. arXiv:2308.08155.